

Infographic

Helena Parsons

2026-06-04

1. Loading packages

```
# loading necessary packages
library(tidyverse)
library(ggthemes)
library(here)
library(corrplot)
library(showtext)
library(ggimage)
library(corr)
```

2. Adding custom fonts and loading ggplot2

```
# choosing font from google library
font_add_google("EB Garamond", "garamond")
# ensures text will be automatically rendered on new devices
showtext_auto()

# loading ggplot2 library
library(ggplot2)
```

3. Visualizations

Reading data

```
# creating new object
job_search <-
  # reading csv from data file
  read_csv(
    # directing function to data file
    here("data",
      # csv: personal-data
      "personal-data.csv"))
```

3a. Visualization #1: Scatterplot

Creating plot

```
# base layer: ggplot
plot_1 <- ggplot(
  # df: job_search
  data = job_search,
  # x-axis : time spent on outside engagements
  # y-axis : time spent on job search
  mapping = aes(x = outside_eng,
    y = job_search)) +

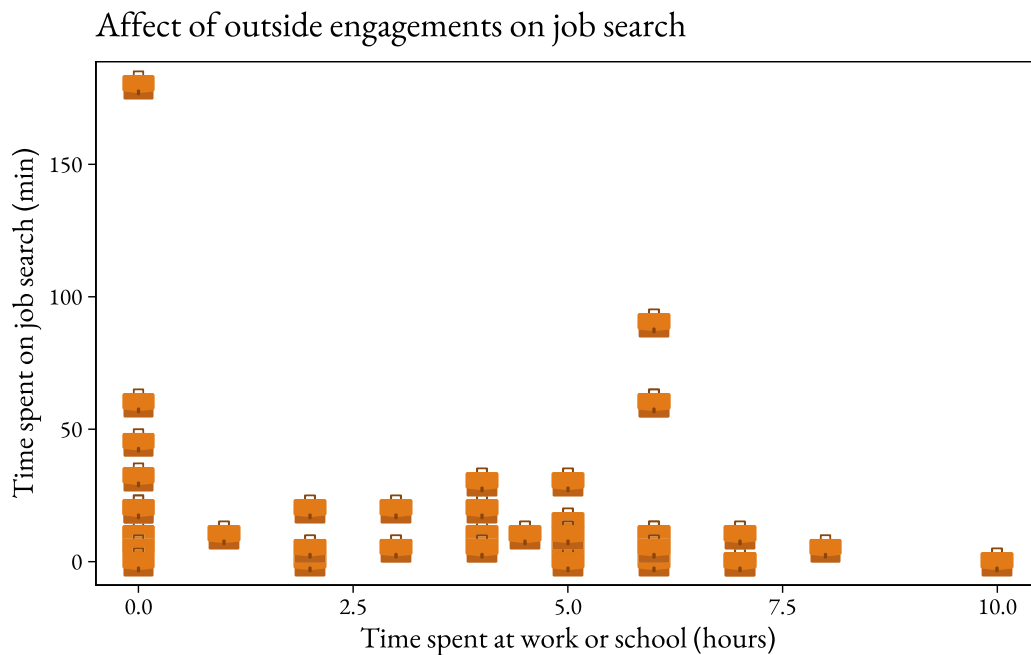
  # first layer: scatterplot
  geom_image(mapping = aes(
    # changing image to briefcase.png
    image = (
      # directing function to images folder
      here("images", "briefcase.png"))),
    # changing size of image
    size = 0.07) +

  # changing title and axes' titles
  labs(title = "Affect of outside engagements on job search",
    x = "Time spent at work or school (hours)",
    y = "Time spent on job search (min)") +

  # customizing theme
  theme_linedraw() +
```

```
# customizing font
theme(text = element_text(family = "garamond")) +
# getting rid of gridlines
theme(panel.grid.major = element_blank(),
      panel.grid.minor = element_blank())

# displaying plot
plot_1
```



```
# saving plot
ggsave(
  # specifying folder and saving as pdf
  "images-to-export/scatterplot_1.pdf",
  # plot: plot_1
  plot = plot_1)
```

3b. Visualization #2: Scatterplot 2 for stress

Creating plot

```

# base layer: ggplot
plot_2 <- ggplot(
  # df: job_search
  data = job_search,
  # x-axis : level of stress
  # y-axis : time spent on job search
  mapping = aes(x = stress,
                  y = job_search)) +

# first layer: scatterplot
geom_image(mapping = aes(
  # changing image to briefcase.png
  image = (
    # directing function to images folder
    here("images", "briefcase.png"))),
  # changing size of image
  size = 0.07) +

# changing title and axes' titles
labs(title = "Affect of stress on job search",
      x = "Level of stress (scale of 1-5)",
      y = "Time spent on job search (min)") +

# customizing theme
theme_linedraw() +

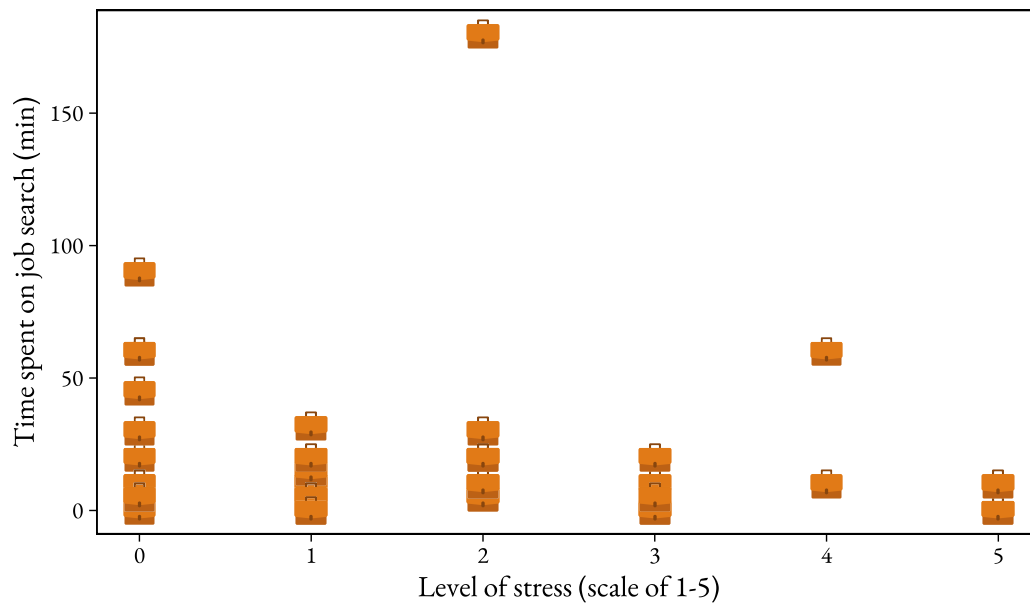
# customizing font
theme(text = element_text(family = "garamond")) +

# getting rid of gridlines
theme(panel.grid.major = element_blank(),
      panel.grid.minor = element_blank())

# displaying plot
plot_2

```

Affect of stress on job search



```
# saving plot
ggsave(
  # specifying folder
  "images-to-export/scatterplot2.pdf",
  # plot: plot_1
  plot = plot_2)
```

3c. Visualization #3: Ridgeline plot

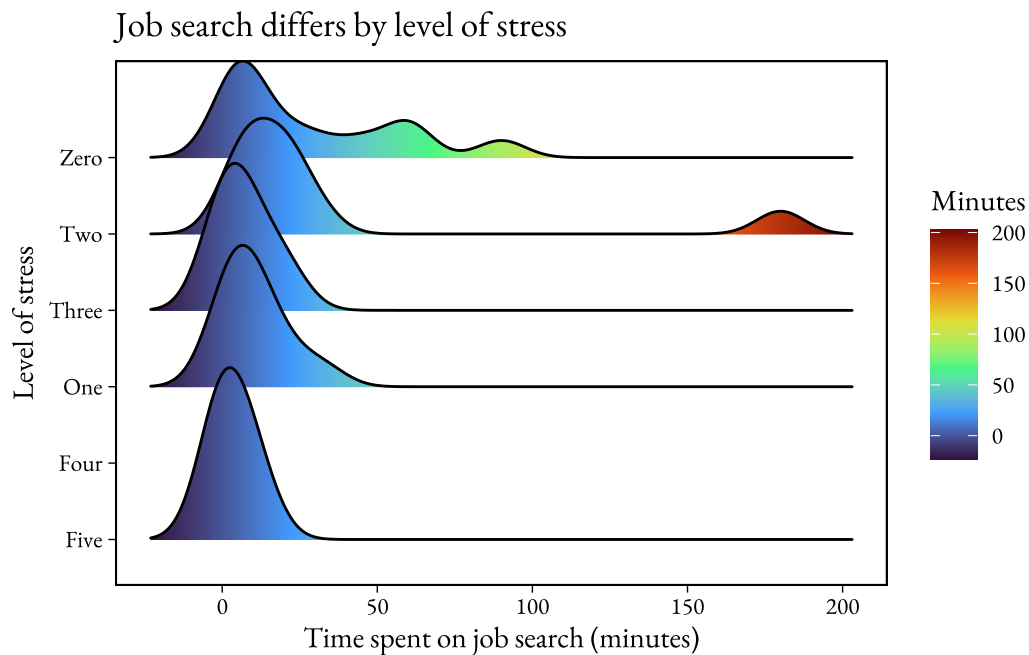
Cleaning and wrangling data

```
# creating new object
job_search_clean <-
  # df: job_search
  job_search |>
  # mutating data
  mutate(
    # changing values to be words in stress column
    stress = case_when (
      stress == "0" ~ "Zero",
      stress == "1" ~ "One",
```

```
stress == "2" ~ "Two",  
stress == "3" ~ "Three",  
stress == "4" ~ "Four",  
stress == "5" ~ "Five"))
```

Creating plot

```
# base layer: ggplot  
plot_3 <- ggplot(  
  # df: job_search_clean  
  data = job_search_clean,  
  # x-axis: time spent on job search  
  # y-axis: level of stress  
  # filling by x-axis value  
  mapping = aes(x = job_search,  
                y = stress,  
                fill = after_stat(x))) +  
  # first layer: ridgeline plot  
  geom_density_ridges_gradient() +  
  # coloring ridgeline by gradient corresponding to x-axis value  
  scale_fill_viridis_c(option = "turbo", name = "Minutes") +  
  # changing title and axes' titles  
  labs(title = "Job search differs by level of stress",  
        x = "Time spent on job search (minutes)",  
        y = "Level of stress"  
  ) +  
  # changing theme  
  theme_linedraw() +  
  # changing font  
  theme(text = element_text(family = "garamond")) +  
  # getting rid of gridlines  
  theme(panel.grid.major = element_blank(),  
        panel.grid.minor = element_blank())  
  
# displaying plot  
plot_3
```



```
# saving plot
ggsave(
  # specifying folder and saving as pdf
  "images-to-export/ridgelinegraph.pdf",
  # plot: plot_3
  plot = plot_3)
```

3d. Visualization #4: Correlation barplot

Cleaning and wrangling data

```
# creating new object
job_search_corr <-
  # df: job_search
  job_search %>%
  # calculating correlation between variables
  correlate() %>%
  # calculating correlation only for response variable
  focus(job_search)

# creating new object
```

```

job_search_corr_clean <-
  # df: job_search_corr
  job_search_corr |>
  # mutating data
  mutate(
    # cleaning names in term column
    term = case_when (
      term == "outside_eng" ~ "Outside engagements",
      term == "stress" ~ "Stress",
      term == "sleep" ~ "Sleep",
      term == "social_media" ~ "Social media use",
      term == "day_to_grad" ~ "Days until graduation"))

```

Creating plot

```

# mutating df
plot_4 <- job_search_corr_clean %>%
  # ordering data by correlation strength to job_search
  mutate(term = factor(term, levels = term[order(job_search)])) %>%
  # creating plot
  # base layer: ggplot
  ggplot(
    # x-axis: term or explanatory variable
    # y-axis: correlation to response variable (job_search)
    aes(x = term, y = job_search)) +
  # first layer: bar plot
  geom_bar(
    # using y-axis as dependent variable
    stat = "identity",
    # changing color of bars
    fill = "turquoise4") +
  # changing title and axes' titles
  labs(title = "Correlation strength is greatest with outside engagements",
    x = "Variable",
    y = "Correlation with time spent on job search") +
  # custom theme
  theme_linedraw() +
  # changing font
  theme(text = element_text(family = "garamond")) +
  # getting rid of gridlines

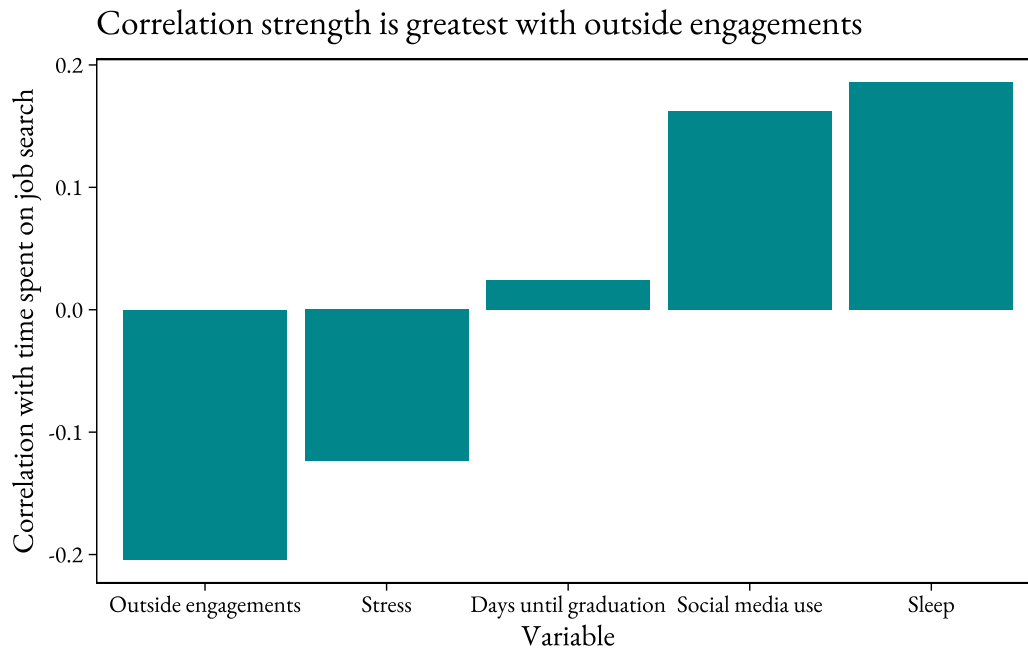
```

```

theme(panel.grid.major = element_blank(),
      panel.grid.minor = element_blank())

# displaying plot
plot_4

```



```

# saving plot
ggsave(
  # specifying folder and saving as pdf
  "images-to-export/barplot.pdf",
  # plot: plot_1
  plot = plot_4)

```

4. Write up

General information about design and visualization

4a. What patterns are you highlighting in each visualization? Why?

First I wanted to highlight that the time spent on my job search appears lowest with an increase in both the time spent on outside engagements and the level of stress. However, to

emphasize what I anecdotally noticed about the effect of stress, as well as how there appears to be less variation in the scatterplot for stress in comparison to the plot for time spent on outside engagements, I decided to create a ridgeline plot for the stress variable. Finally, I wanted to visualize which variable actually had the strongest correlation to the response variable, so I created a correlation barplot.

4b. Outside of the data visualization, what aesthetic choices (e.g. color, font, arrangement) did you make and why? In what way do your choices contribute to a compelling or interesting narrative?

For the first two scatterplots, I was inspired by the feedback I got in workshop on my affective visualization, and I used the “ggimage” package to use briefcase icons to represent the points. I feel like that allows the viewer to connect the data to the response variable it actually represents, which is the time I spent searching for jobs. Additionally I wanted to choose a new font that was elegant but simple, and would enhance the aesthetics of the plot without being too loud, so I used the “showtext” package to use the “EB garamond” font in all the plots.

Sources and process

4c. For each visualization, describe what examples inspired that visualization (cite the author)

For the first two visualizations, I was inspired by “The Chonk Index: Palmer Penguins’ Body Mass,” plot created by Ijeamaka Anyene, where icons of different penguin species are used to represent their differences in body mass.]

For my ridgeline plot, I was inspired by Cedric Sherer’s visualization of the bill ratio of different penguin species from the Palmer Penguins dataset.

For my correlation barplot, I was inspired by the Manasseh Oduor’s visualization of attendance patterns by university, although they used a violin plot and a categorical variable, but I liked how it clearly displayed how one relationship between variables stood out against the others.

4d. Describe your coding process, for example (not the only options for this prompt): did you have to clean up/summarize the data before you started? Which geoms did you start with? What geoms did you end up using, and why?

For the first two visualizations, I had to create an images folder and upload the briefcase image I wanted to use to represent the points directly into my github directory, and then clone the repository to Rstudio. For the ridgeline plot, I had to clean up the data by changing the values of the stress column from numbers to words, to allow the geom_density_ridges_gradient function to register the y-axis as categorical data. Finally, for the correlation barplot I had

to create a new object to determine the correlation between the response variable with each explanatory variable, and then I created the barplot and ordered it from most negative to most positive so that it was more intuitive to understand.

4e. Describe the tools you used: other people's code? Google/StackOverflow? ChatGPT (with examples of prompts)?

I used google, youtube videos, and websites such as the r-graph-gallery and StackOverflow to help me figure out how to code the plots I was unfamiliar with and use the “ggimage” package. I also used old workshop and homework assignments to help me remember how to customize plots so that they fit the overall aesthetic of the infographic better. StackOverflow was extremely helpful with creating the correlation barplot, as it provided clear examples with annotations on how to mutate and visualize the data.